

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ОДЕСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ І.І.МЕЧНИКОВА
Факультет математики, фізики та інформаційних технологій
Кафедра математичного забезпечення комп'ютерних систем

КУРСОВИЙ ПРОЄКТ
з освітнього компоненту «Програмування»
на тему: СТВОРЕННЯ ІЄРАРХІЇ КЛАСІВ НА ТЕМУ
«ТЕЛЕФОННИЙ ДОВІДНИК»

Студента 1 курсу, денна форма навчання
спеціальність F7 «Комп'ютерна інженерія»

Коробова Олександра Володимировича

Керівник: к.ф.- м.н., доц. Антоненко О.С.

Національна шкала: _____

Кількість балів: _____

ECTS: _____

Дата захисту: _____

Члени комісії _____ Антоненко О.С.
(підпис) (прізвище та ініціали)

_____ Шестопапов С.В.
(підпис) (прізвище та ініціали)

_____ _____
(підпис) (прізвище та ініціали)

ЗМІСТ

	стор.
ПОСТАНОВКА ЗАВДАННЯ	3
1. ТЕОРЕТИЧНІ ВІДОМОСТІ.....	4
2. СТРУКТУРА ПРОГРАМИ.....	5
2.1 Клас «PhoneBook»	5
2.2 Абстрактний клас «Record».....	6
2.3 Клас «Person»	7
2.4 Клас «Organization»	7
2.5 Клас «Phone»	8
2.6 Опис деяких функцій	9
3. ІНТЕРФЕЙС	10
ВИСНОВКИ.....	13
СПИСОК ЛІТЕРАТУРИ.....	14
ДОДАТОК А Діаграма класів.....	15
ДОДАТОК Б Методи класу «PhoneBook».....	16
ДОДАТОК В Код класу «Record»	18
ДОДАТОК Г Код класу «Phone»	19

ПОСТАНОВКА ЗАВДАННЯ

Студент повинен розробити консольний додаток, аналог «Телефонного довідника», мовою C++. Розроблений додаток повинен містити в собі записи (Records) та виконувати такі функції: пошук людини або організації за номером телефону, пошук за електронною поштою; пошук людини за ПІБ, частиною ПІБ; додавання та видалення записів в довіднику; виведення вмісту довідника в консоль. Довідник повинен мати інтерактивний, охайний та інтуїтивно зрозумілий інтерфейс. Проєкт повинен бути написаний з дотриманням принципів ООП, включаючи інкапсуляцію, успадкування, поліморфізм. Проєкт має містити чітку ієрархію класів. Необхідно використовувати вбудовані класи C++, такі як `string`, `vector`, і т. д.

В додатку мають бути реалізовані такі класи:

- a) довідник – головний клас, містить динамічний масив усіх записів, повинен дозволяти виконувати основні функції, такі як додавання або видалення запису, пошук запису;
- b) запис у довіднику – батьківський абстрактний клас, містить масив номерів телефонів та масив електронних адрес, використовується як основа під записи. Так як клас є абстрактним, повинен містити віртуальні методи;
- c) людина – клас-нащадок, повинен містити загальну інформацію про людину, вказівник на організацію (якщо є) та успадковувати номери телефонів та електронні адреси від батьківського класу;
- d) організація – клас-нащадок, повинен містити загальну інформацію про організацію, вказівник на керівника (якщо є), та успадковувати номери телефонів та електронні адреси;
- e) телефон – окремий клас, містить три категорії (домашній, мобільний та робочий) та три форми (повна, середня та коротка).

1. ТЕОРЕТИЧНІ ВІДОМОСТІ

Об'єктно-орієнтоване програмування (ООП) – це сукупність методів програмування, що вбачає програму як сукупність об'єктів, що взаємодіють між собою [2]. Воно виникло як протидія ускладненим процесам розробки великого програмного забезпечення, і наразі використовується майже всіма. В ООП кожен об'єкт – це екземпляр класу, а кожен клас є каркасом, який задає стан та поведінку об'єкта. ООП поширює такі принципи:

- а) інкапсуляція – приховування внутрішньої реалізації об'єкта та захищення доступу до його даних від випадкових змін. З об'єктом взаємодія повинна відбуватись виключно через публічні методи, які дозволяють виводити або змінювати інформацію в межах класу [1];
- б) наслідування – механізм, який дозволяє створювати нові класи на основі існуючих. У ньому нащадок успадковує властивості та методи батьківського класу, що робить код більш компактним [3];
- с) поліморфізм – дозволяє об'єктам різних класів з однаковим інтерфейсом виконувати дії по-різному.

Блискучість ООП полягає в спрощенні структури коду до окремих складових, що значно полегшує розуміння, тестування та особливо масштабування коду.

2. СТРУКТУРА ПРОГРАМИ

2.1 Клас «PhoneBook»

Об'єкт цього класу – наш довідник. Він є кінцевою точкою кожного поля даних програми, та найголовніший у ієрархії, яку детально представлено в додатку А. Уся інформація довідника збирається і сортується тут. Це той клас, з яким користувач працює напряму.

Довідник – набір записів, отже наш клас не містить нічого окрім самих записів. Це зручно для тих задач, де потрібно проаналізувати весь довідник з початку до кінця, а не його окремі елементи. В «PhoneBook» будуть реалізовані методи пошуку, додавання та вилучення інформації. Код методів класу представлений в додатку Б.

Поля:

- a) `std::vector<Record*> records` – приватне поле, набір посилань на усі записи довідника.

Методи:

- a) `~PhoneBook()` – деструктор, видаляє кожен елемент `records` по черзі.
- b) `void addRecord(Record* record)` – додає в кінець масиву `records` посилання на `record`;
- c) `void removeRecord(int index)` – приймає числове значення, видаляє запис із масиву за цим індексом, обов'язково перевіряє чи існує елемент з цим індексом;
- d) `std::vector<Record*> findByName(const std::string& name)`
`const` – повертає список посилань на записи, в яких виявлено задане ім'я. Метод одразу підходить і для повних ПІБ, і для їх частин;
- e) `std::vector<Record*> findByPhone(const std::string& phoneNumber)` `const` – повертає масив посилань на записи, в яких

виявлено заданий номер телефону. Одночасно перевіряє співпадіння повних, середніх та коротких форм;

- f) `std::vector<Record*> findByEmail(const std::string& searchEmail)` `const` – повертає масив посилань на записи зі знайденими адресами.

Кожен пошуковий метод використовує гетери нащадків `Records` для порівняння шуканої інформації з приватними полями об'єктів. Пізніше отримання порожніх векторів буде інтерпретовано як відсутність співпадінь.

2.2 Абстрактний клас «Record»

Складова нашого довідника. Містить в собі усю інформацію про запис, доповнену класами нащадками, окрім свого розміщення у списку довідника. Слугує як базис для двох типів записів: людей і організацій, адже їх об'єднують однакові ролі та функції, а також наявність номерів телефонів та електронних адрес. Код класу детальніше показаний в додатку В.

Поля:

- a) `std::vector<Phone> phones` – список номерів телефонів;
 b) `std::vector<std::string> emails` – список email'ів;

Методи:

- a) конструктор з параметрами;
 b) `virtual ~Record()` – віртуальний деструктор;
 c) `virtual void print() const` – віртуальний метод роздруківки, який буде реалізовуватись в нащадках;
 d) Гетери для масивів «phones» та «emails».

2.3 Клас «Person»

Будучи нащадком «Record», «Person» виділяється наявністю ПІБ, статі та вказівника на організацію. Так як клас «Organization» поки що не заданий, використаємо попереднє оголошення класу.

Поля:

- a) `std::string name` – ПІБ (або будь-яке ім'я, так як це ні на чого не впливає);
- b) `std::string gender` – стать (не обмежуємо варіантами, так як це також ні на що не впливає);
- c) `Organization* organization` – вказівник на організацію (повинна існувати в довіднику).

Методи:

- a) конструктор з параметрами;
- b) гетери `std::string getName() const` та `Organization* getOrganization() const`;
- c) сетер `void setOrganization(Organization* org)`;
- d) `void print() const` – виводить на екран всю інформацію про людину. Якщо вказівник на організацію нульовий – виводить «немає» замість назви організації. Виводить кожний номер телефону по черзі, вказуючи категорію та усі його формати. Перший номер телефону – головний.

2.4 Клас «Organization»

На відміну від «Person», має назву організації та її вид діяльності, а також вказівник на керівника. Знову використовуємо попереднє оголошення класу.

Поля:

- a) `std::string title` – назва;
- b) `std::string activity` – діяльність;
- c) `Person* manager` – вказівник на керівника (повинен існувати в довіднику);

Методи:

- a) конструктор з параметрами;
- b) гетери `std::string getTitle() const` та `Person* getManager() const`;
- c) сетер `void setManager (Person* mngr);`
- d) `void print() const` – виводить на екран всю інформацію про організацію. Якщо вказівник на керівника нульовий – виводить «немає». Виводить кожний номер телефону по черзі, вказуючи категорію та усі його формати. Перший номер телефону – головний.

2.5 Клас «Phone»

Номер телефону – об’єкт, який має два параметри: категорія та сам номер. Тому зручніше створити клас «Phone» і в ньому задати метод `print()`, який буде охайно виводити інформацію про нього. Подивитися на код класу можна в додатку Г.

Поля:

- a) `std::string category`;
- b) `std::string full_form`;
- c) `std::string medium_form`;
- d) `std::string short_form`.

Методи:

- a) `std::string getCategory() const`;
- b) `std::string getFull() const`;

- c) `std::string getMedium() const;`
- d) `std::string getShort() const;`
- e) конструктор з параметрами;
- f) `void print() const` – роздруківка.

Усі поля класів знаходяться в приватному доступі. Поля класу «Record» – у захищеному.

2.6 Опис деяких функцій

`std::string choosePhoneCategory()` – використовується для того, щоб дати користувачу вибір між трьома категоріями номерів. Її особливість: вибір повинен бути зроблений, інакше програма не дозволить перейти до наступного кроку. Передбачено вибір неіснуючих варіантів, що виходять за межі одиниці та трійки.

`std::vector<Phone> enterPhones()` – дозволяє вводити будь-яку кількість телефонів в запис довідника. Використовується під час додавання запису. Перевіряє введений номер телефону на правильність та одразу переводить заданий номер у його середню та коротку форми. Щоб завершити функцію, або взагалі не додавати номери, користувач повинен ввести порожній рядок.

`std::vector<std::string> enterEmails()` – аналогічно до `enterPhones`, однак формат адреси не обмежується конкретними правилами.

Інтерфейс реалізується за допомогою команди `switch()`. Користувачу висвітлюється меню, яким він керує за допомогою введення цифр.

3. ІНТЕРФЕЙС

Програма працює в циклі `while(true)`. Щоб вийти з програми, користувач повинен натиснути «0» в головному меню, тоді буде викликано `return 0`.

При запуску програми висвітлюється головне меню (рис 3.1).

```

Вітаю в телефонному довіднику.
Оберіть дію:
1. Вивести довідник.
2. Додати запис.
3. Видалити запис.
4. Пошук запису.
0. Вийти.
>>> 

```

Рисунок 3.1 – Головне меню

Перший варіант виводить всі записи в довіднику по черзі, позначаючи їх своїм номером (рис. 3.2).

```

Оберіть дію:
1. Вивести довідник.
2. Додати запис.
3. Видалити запис.
4. Пошук запису.
0. Вийти.
>>> 1
(1)
Назва: Київстар
Діяльність: Готель
Керівник: немає
Головний номер телефону: Мобільний: +380506523594 (0506523594, 652-3594)
Домашній: +380638563973 (0638563973, 856-3973)
Домашній: +380679259639 (0679259639, 925-9639)
Домашній: +380674693784 (0674693784, 469-3784)
Електронні пошти:
contact@gmail.com
contact@gmail.com
-----
(2)
Назва: Київстар
Діяльність: Взуття
Керівник: немає
Головний номер телефону: Мобільний: +380630308034 (0630308034, 030-8034)
Електронні пошти:
support@thisandthat.com
support@thisandthat.com
-----
(3)
Назва: Пузата Хата
Діяльність: Продажі

```

Рисунок 3.2 – Фрагмент довідника

Другий варіант дозволяє додати запис в довідник. Спочатку програма питає, чи хоче користувач додати людину, чи організацію. Реалізується також за допомогою `switch()`. «0» викликає `break` та повертає до головного меню (рис 3.3).

```
Стать: жіноча
Організація: немає
Головний номер телефону: Робочий: +380636567231 (0636567231, 656-7231)
Робочий: +380938465796 (0938465796, 846-5796)
Електронні пошти:
contact@gmail.com
office@kahoot.com
-----
Оберіть дію:
1. Вивести довідник.
2. Додати запис.
3. Видалити запис.
4. Пошук запису.
0. Вийти.
>>> 2
1. Додати людину
2. Додати компанію.
0. Скасувати
```

Рисунок 3.3 – Вибір типу запису

Після вибору, користувач повинен ввести по черзі інформацію про новий запис (рис 3.4). Його буде додано в кінець довідника.

```
>>> 2
1. Додати людину
2. Додати компанію.
0. Скасувати
1
Введіть ПІБ: Іваненко Іван Іванович
Введіть стать: чоловіча
Введіть організацію (якщо є): Neskafe
Організація 'Neskafe' не знайдена. Спробуйте ще раз.
Введіть організацію (якщо є): Anuker
Введіть номер телефону (або пустий рядок щоб завершити, формат +380XXXXXXXX): +380774488995
Оберіть категорію номера:
1. Домашній
2. Мобільний
3. Робочий
2
Введіть номер телефону (або пустий рядок щоб завершити, формат +380XXXXXXXX):
Введіть email (або пустий рядок щоб завершити): gold@goldgold.com
Введіть email (або пустий рядок щоб завершити):
Запис додано.
Оберіть дію:
1. Вивести довідник.
2. Додати запис.
3. Видалити запис.
4. Пошук запису.
0. Вийти.
>>> █
```

Рисунок 3.4 – Процес створення запису

Третій варіант дозволяє видалити запис під відповідним номером. Користувач знову має змогу ввести «0» щоб повернутися в головне меню.

Четвертий варіант викликає меню пошуку. Користувач повинен обрати метод, за яким буде шукати записи (рис 3.5). Потім програма виведе усі записи, що підійшли під пошук (рис 3.6).

```
Оберіть дію:
1. Вивести довідник.
2. Додати запис.
3. Видалити запис.
4. Пошук запису.
0. Вийти.
>>> 4
Оберіть критерій пошуку:
1. Пошук за іменем
2. Пошук за номером телефону
3. Пошук за email
0. Скасувати
█
```

Рисунок 3.5 – Вибір критерію пошуку

```
1
Введіть ім'я для пошуку: Андрій
ПІБ: Сукачук Андрій Миколайович
Стать: чоловіча
Організація: BasedBodyworks
Головний номер телефону: Мобільний: +380673374756 (0673374756, 337-4756)
Електронні пошти:
info@baza.ua
support@thisandthat.com
-----
Оберіть дію:
1. Вивести довідник.
2. Додати запис.
3. Видалити запис.
4. Пошук запису.
0. Вийти.
>>> █
```

Рисунок 3.6 – Знайдені записи

ВИСНОВКИ

Створено телефонний довідник, який має змогу виконувати всі необхідні за умовою завдання функції. Реалізовано інтерактивне керування програмою в терміналі та розбиття проєкту на ієрархію класів. Застосовано основні принципи ООП на практиці.

ООП допомагає розбити складне завдання на його прості складові. Такий формат дозволяє чітко бачити кожен компонент програми і за що він відповідає. Ієрархія класів – невід’ємна частина ООП, яка дозволяє створювати зв’язки між об’єктами та даними без плутанини. Наш телефонний довідник – це приклад, як можна купу рядків, типів та задач організовувати в окремі елементи програми і керувати кожним з них від їх завдання до перетворення на потрібний результат.

Телефонний довідник чудово демонструє структуру об’єктів та методів в ООП. Кожен довідник – це сукупність записів. Кожен запис розділяється на дві категорії. Кожна категорія має однакове призначення – виведення чи запис інформації, але різні категорії зберігають різну інформацію. Інформація в записах також розбивається на класи, такі як Phone, string, vector, ці класи в свою чергу мають своє призначення і зберігають іншу інформацію. Всі ці класи збираються в один довідник, щоб над ними виконувалися дії. А якщо потрібно змінити щось для окремого класу, чи додати новий клас або поле, інші не потрібно підлаштовувати під зміни.

СПИСОК ЛІТЕРАТУРИ

1. Страуструп Б. Програмування: принципи та практика використання C++ : пер. з англ. / Б. Страуструп. – 2-ге вид. – Київ : Вільямс, 2017. – 1328 с.
2. Мейєрс С. Ефективний C++. 55 специфічних способів покращити програми та проекти : пер. з англ. / С. Мейєрс. – Львів : Видавництво Старого Лева, 2021. – 320 с.
3. Лаfore Р. Об'єктно-орієнтоване програмування в C++ : пер. з англ. / Р. Лаfore. – Харків : Фабула, 2020. – 928 с.

ДОДАТОК А

Діаграма класів

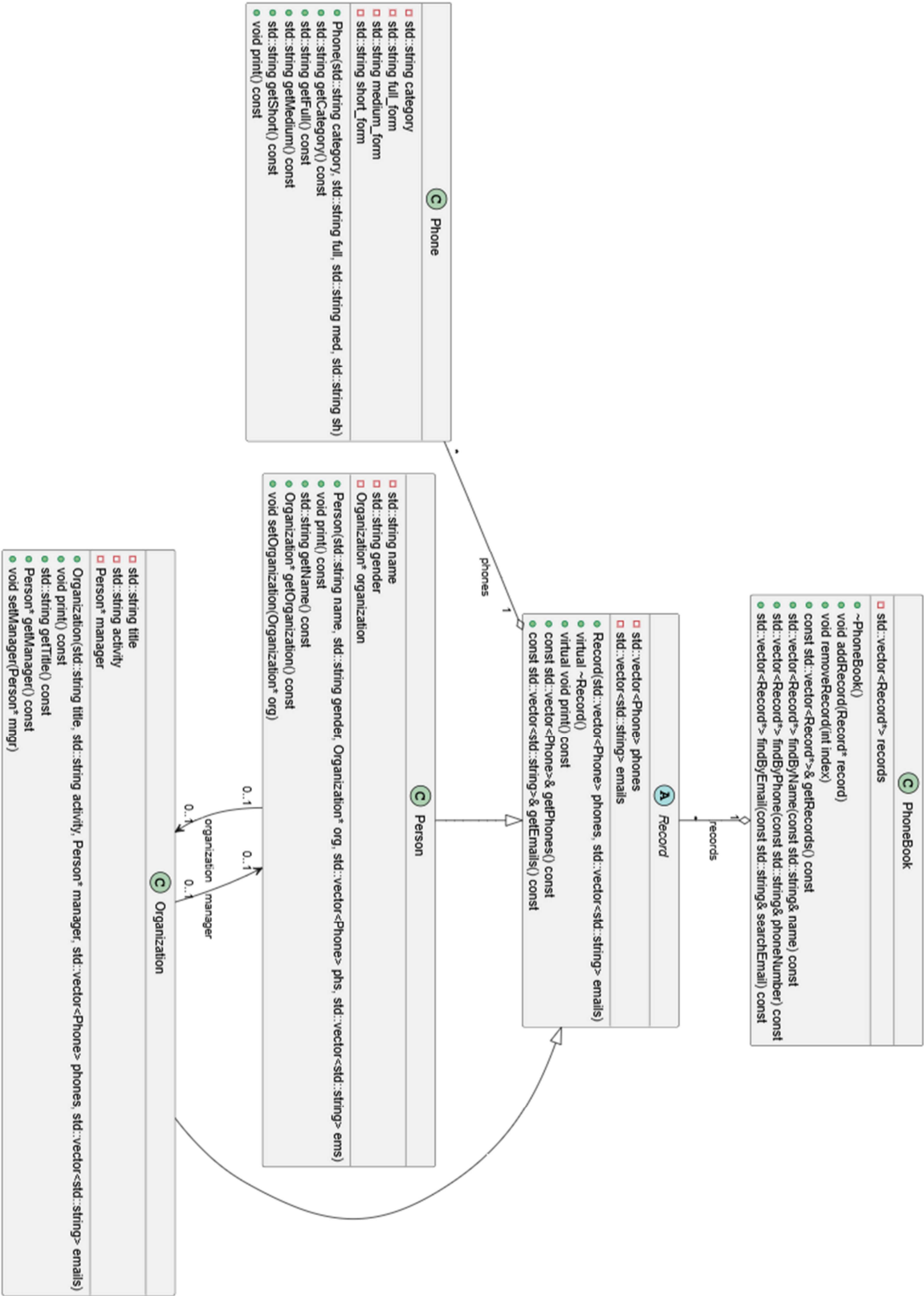


Рисунок А.1 – Діаграма класів

ДОДАТОК Б

Методи класу «PhoneBook»

```

void PhoneBook::removeRecord(int index){
    if (index < 0 || index >=
        static_cast<int>(records.size())) {
        return;
    }
    Record* record = records[index];

    if (auto* org = dynamic_cast<Organization*>(record)) {
        for (auto* rec : records) {
            if (auto* person = dynamic_cast<Person*>(rec)) {
                if (person->getOrganization() == org) {
                    person->setOrganization(nullptr);
                }
            }
        }
    } else if (auto* person = dynamic_cast<Person*>(record)) {
        for (auto* rec : records) {
            if (auto* org = dynamic_cast<Organization*>(rec))
        {
                if (org->getManager() == person) {
                    org->setManager(nullptr);
                }
            }
        }
    }
}

```

Лістинг Б.1 – Метод видалення запису

```

std::vector<Record*> PhoneBook::findByName(const std::string&
name) const{
    std::vector<Record*> found;
    for(auto record : records){
        Person* check = dynamic_cast<Person*> (record);
        if(check != nullptr && (check->getName()).find(name)
!= std::string::npos){ found.push_back(record); }
    } return found; }

```

Лістинг Б.2 – Метод пошуку за іменем


```

std::vector<Record*> PhoneBook::findByPhone(const std::string&
phoneNumber) const{
    std::vector<Record*> found;
    for(auto record : records){
        for (auto phone : record->getPhones()){
            if(phoneNumber == phone.getFull() || phoneNumber
== phone.getMedium() || phoneNumber == phone.getShort()){
                found.push_back(record);
                break;
            }
        }
    }
    return found;
}

```

Лістинг Б.3 – Метод пошуку за номером телефону

```

std::vector<Record*> PhoneBook::findByEmail(const std::string&
searchEmail) const {
    std::vector<Record*> found;
    for (auto record : records){
        for(auto email : record->getEmails()){
            if (email == searchEmail){
                found.push_back(record);
                break;
            }
        }
    }
    return found;
}

```

Лістинг Б.4 – Метод пошуку за електронною поштою

ДОДАТОК В

Код класу «Record»

```
class Record {
    protected:
        std::vector<Phone> phones;
        std::vector<std::string> emails;
    public:
        Record(std::vector<Phone> phones,
std::vector<std::string> emails)
            : phones(phones), emails(emails){}

        virtual ~Record() = default;

        virtual void print() const = 0;

        const std::vector<Phone>& getPhones() const { return
phones; }
        std::vector<Phone>& getPhones() { return phones; }
        const std::vector<std::string>& getEmails() const {
return emails;}
        std::vector<std::string>& getEmails() { return emails;
}
};
```

Лістинг В.1 – Клас «Record»

ДОДАТОК Г

Код класу «Phone»

```

class Phone {
    private:
        std::string category;
        std::string full_form;
        std::string medium_form;
        std::string short_form;
    public:
        Phone(std::string category, std::string full,
std::string med, std::string sh)
            : category(category), full_form(full),
medium_form(med), short_form(sh) {}

        std::string getCategory() const { return category; }
        std::string getFull() const { return full_form; }
        std::string getMedium() const { return medium_form; }
        std::string getShort() const { return short_form; }

        void print() const {
            std::cout << category << ": "
                << full_form
                << " (" << medium_form << ", "
                << short_form << ")";
        }
};

```

Лістинг Г.1 – Клас «Phone»